

Министерство науки и высшего образования Российской
Федерации



Федеральное государственное бюджетное
образовательное учреждение высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления»

КАФЕДРА «Программное обеспечение ЭВМ и информационные технологии»

Лабораторная работа № 5 по дисциплине «Анализ алгоритмов»

Тема Организация параллельных вычислений по конвейерному принципу

Студент Пантелеев Василий Максимович

Группа ИУ7-52Б

Преподаватели Строганов Д. В. Волкова Л. Л.

Москва, 2024

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	3
1 Входные и выходные данные	3
2 Преобразование входных данных в выходные	4
3 Тестирование	4
4 Описание исследования	4
4.1 Технические характеристики	4
4.2 Результаты исследования	5
ЗАКЛЮЧЕНИЕ	7
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	8

ВВЕДЕНИЕ

Параллельные вычисления по конвейерному принципу представляют собой подход, при котором обработка задачи разбивается на несколько независимых этапов (стадий), выполняемых последовательно, но параллельно. Каждая стадия обрабатывает свою часть данных, передавая результаты на следующую стадию через общие очереди. Это позволяет эффективно использовать многопоточность, минимизируя простой процессоров за счёт равномерного распределения нагрузки между потоками [1].

Нативные потоки (Native Threads) — это потоки, управляемые непосредственно операционной системой. Использование нативных потоков позволяет создавать высокопроизводительные многопоточные приложения, но требует контроля доступа к общим ресурсам для предотвращения гонок данных.

Цель работы — получение навыка организации параллельных вычислений по конвейерному принципу.

Для достижения поставленной цели необходимо выполнить следующие задачи:

- описать входные и выходные данные;
- анализ структуры сайта;
- разработка ПО, выполняющего парсинг рецептов в конвейере и загрузку информации в базу данных;
- исследование характеристик разработанного ПО на данных о интервалах времени стадий обработки рецепта.

1 Входные и выходные данные

Входными данными программы являются скаченные страницы рецептов с ресурса [2]. Выходными данными является файлы json – загруженные рецепты по ссылкам из входных данных, при этом в каждом json файле хранится:

- id задачи;
- название рецепта;

- список ингредиентов;
- шаги рецепта.

2 Преобразование входных данных в выходные

Входные данные, представленные в виде текста страницы рецепта, проходят три этапа обработки. На первом этапе выполняется чтение страницы из файла, где извлекается её содержимое. На втором этапе текст анализируется: выделяются URL рецепта, название, ингредиенты (с указанием названия, количества и единицы измерения), шаги приготовления и URL изображения. Эти данные очищаются от HTML-тегов и других лишних символов. На третьем этапе обработанная информация преобразуется в формат, пригодный для хранения в базе данных (JSON для ингредиентов и шагов) и записывается в соответствующие поля таблицы.

3 Тестирование

Тестирование программы проводилось подачей на вход директории с 60 файлами со страницами различных рецептов веб-ресурса [2]. При этом отслеживалось количество удачных скачиваний и обработок страницы, а также выборочная ручная проверка 5 случайных выходных файлов.

4 Описание исследования

В ходе исследования требуется сформировать лог обработки задач.

4.1 Технические характеристики

Технические характеристики используемого устройства:

- процессор: AMD Ryzen™ 7–5800H with Radeon™ Graphics × 16;
- ОС: 64-разрядная операционная система Ubuntu 24.04.1 LTS;
- оперативная память: 16,0 ГБ;

- число логических ядер – 16.

4.2 Результаты исследования

В таблице 4.1 приведен фрагмент лога обработки. Обозначения событий:

- start_step_1 — начало обработки на стадии 1;
- end_step_1 — окончание обработки на стадии 1;
- start_step_2 — начало обработки на стадии 2;
- end_step_2 — окончание обработки на стадии 2;
- start_step_3 — начало обработки на стадии 3;
- end_step_3 — окончание обработки на стадии 3.

Таблица 4.1 – Фрагмент лога обработки (начало)

Метка времени, мкс	Событие	ID записи
1732747886220541	start_step_1	32
1732747886220575	end_step_1	32
1732747886220593	start_step_1	33
1732747886220628	end_step_1	33
1732747886220644	start_step_1	34
1732747886220680	end_step_1	34
1732747886220700	start_step_1	35
1732747886220732	end_step_1	35
1732747886220752	start_step_1	36
1732747886220785	end_step_1	36
1732747886220806	start_step_1	37
1732747886220842	end_step_1	37
1732747886220862	start_step_1	38
1732747886220898	end_step_1	38
1732747886220920	start_step_1	39
1732747886220954	end_step_1	39
1732747886240923	end_step_2	1

Таблица 4.1 – Фрагмент лога обработки (окончание)

Метка времени, мкс	Событие	ID записи
1732747886240957	start_step_2	2
1732747886241016	start_step_3	1
1732747886243748	end_step_3	1

В ходе исследования получены следующие характеристики:

- среднее время жизни задачи – 448332 мкс;
- среднее время выполнения задачи 1 – 49 мкс;
- среднее время выполнения задачи 2 – 21540 мкс;
- среднее время выполнения задачи 3 – 1959 мкс;
- среднее время ожидания в очереди после шага 1 – 423000 мкс;
- среднее время ожидания в очереди после шага 2 – 51 мкс;
- среднее время ожидания в очереди после шага 3 – 49 мкс.

При конвейерной многопоточной обработке сложность выполнения задачи и время ожидания в очереди тесно взаимосвязаны. Увеличение сложности задачи, требующей больше времени на выполнение, неизбежно приводит к увеличению времени ожидания последующих задач в очередях. Это связано с тем, что более сложные этапы обработки становятся узкими местами в конвейере, замедляя прохождение задач через систему.

ЗАКЛЮЧЕНИЕ

Многопоточная конвейерная обработка является эффективным методом организации параллельных вычислений, особенно для задач, требующих последовательной обработки данных через несколько этапов. Основное преимущество этого подхода заключается в том, что каждую стадию обработки можно выполнять независимо в отдельных потоках, что позволяет повысить производительность за счёт параллельной работы.

Цель работы была достигнута. Решены все поставленные задачи:

- описаны входные и выходные данные;
- разработано ПО, выполняющее парсинг рецептов в конвейере и загрузку информации в базу данных;
- выполнено исследование характеристик разработанного ПО на данных о интервалах времени стадий обработки рецепта.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Уильямс Э. Параллельное программирование на C++ в действии. Практика разработки многопоточных программ [Электронный ресурс]. — 2012. — Режим доступа: <https://dodo.inm.ras.ru/konshin/НПС/bib/НПС-схх11-book.pdf> (дата обращения: 6.11.2024).
2. Кулинарные рецепты [Электронный ресурс]. — Режим доступа: <https://sytyikrolik.ru/> (дата обращения: 3.11.2024).